



Ordenação em Python



Eduardo Ogasawara

<https://eic.cefet-rj.br/~eogasawara>



Criando Vetores de Dados

Vamos criar três vetores: peso, altura e identificador do sujeito. Em Python, dados numéricos são representados por arrays do NumPy.

```
import numpy as np
import pandas as pd

weight = np.array([60, 72, 57, 90, 95, 72])
height = np.array([1.75, 1.80, 1.65, 1.90, 1.74, 1.91])
subject = ["A", "B", "C", "D", "E", "F"]
```

weight

Pesos em kg

height

Alturas em metros

subject

Identificadores

Construindo um DataFrame

Quando dados estão relacionados, usamos um DataFrame. Cada linha representa um sujeito e cada coluna uma variável.

```
d = pd.DataFrame({  
    "weight": weight,  
    "height": height,  
    "subject": subject  
})
```

weight	height	subject
60	1.75	A
72	1.80	B
57	1.65	C
90	1.90	D
95	1.74	E

Ordenar Valores vs Obter a Ordem

np.sort

Devolve os valores ordenados

```
array([1.65, 1.74, 1.75,  
       1.80, 1.90, 1.91])
```

np.argsort

Devolve os índices da ordenação

```
array([2, 4, 0, 1, 3, 5])
```

📌 np.sort reorganiza apenas um vetor. np.argsort mostra as posições que produzem essa ordenação.

Interpretando argsort

Posição Original

Indica a posição original do próximo menor valor

Reorganização

Explica como o vetor foi reorganizado

Reprodução

Índices úteis para reproduzir a ordem

```
idx = np.argsort(height)
```

```
print(idx)
```

```
# Saída: [2 4 0 1 3 5]
```

Ordenando uma Tabela Completa

Em pandas usamos `sort_values` para ordenar tabelas. As linhas são reorganizadas com base em uma coluna e todas as colunas permanecem alinhadas.

```
ds = d.sort_values("height")  
print(ds)
```

	weight	height	subject
2	57	1.65	C
4	95	1.74	E
0	60	1.75	A
1	72	1.80	B
3	90	1.90	D
5	72	1.91	F

O índice original é preservado: é a mesma ordem dada por `np.argsort`. Use `.reset_index(drop=True)` para renumerar as linhas.

Ordenação Crescente e Decrescente

Crescente (padrão)

```
d.sort_values("height",  
              ascending=True)
```

Do menor para o maior valor

Basta usar o parâmetro `ascending` para alternar entre crescente e decrescente.

Decrescente

```
d.sort_values("height",  
              ascending=False)
```

Do maior para o menor valor

Ordenação por Múltiplas Colunas

Critério Primário

Ordena por peso

Critério Secundário

Altura decrescente como desempate

Hierárquico

Ordenação em níveis, cada uma com sua direção

```
d.sort_values(["weight", "height"],
              ascending=[True, False])
```

	weight	height	subject
2	57	1.65	C
0	60	1.75	A
5	72	1.91	F
1	72	1.80	B
3	90	1.90	D
4	95	1.74	E

B e F empatam em 72 kg; a altura decide a ordem entre eles (F, mais alto, vem primeiro).

Conceitos-Chave

np.sort

Vetores são ordenados com np.sort

```
np.sort(height)
```

np.argsort

Índices de ordenação com np.argsort

```
np.argsort(height)
```

sort_values

Tabelas ordenadas com DataFrame.sort_values

```
d.sort_values("height")
```

Alinhamento

Todas as colunas permanecem alinhadas

Referências

Think Python

Downey, A. *Think Python: How to Think Like a Computer Scientist*. O'Reilly Media.

Introdução essencial à programação e pensamento computacional com Python.

Python Data Science Handbook

VanderPlas, J. *Python Data Science Handbook*. O'Reilly Media.

Guia abrangente para NumPy, Pandas e bibliotecas de visualização em Python.

Data Science from Scratch

Grus, J. *Data Science from Scratch*. O'Reilly Media.

Aprenda fundamentos de ciência de dados construindo algoritmos do zero com Python.